# diabetes_analysis.R

Vasilis Chatzitolios

```r
################################################################################
# FINAL PROJECT SCENARIO 1: UNDERSTANDING DIABETES PROGRESSION
#
# Analysis for the final project portfolio (Erasmus+ LEAP course, AUTH eLearning).
# Author: Vasilis Chatzitolios
#
# The script runs from top to bottom, from the repository root or from analysis/.
# It is adapted from the supplied starter script (assignment/starter_script.R).
# Final figures are saved to figures/.
################################################################################

# ---------------------------------------------------------------------------- #
# 0. SETUP
# ---------------------------------------------------------------------------- #

# The project uses a small number of common packages.
# We use requireNamespace() to check whether each package is available.
# If a package is missing, R will install it before loading it.
needed_packages <- c("ggplot2", "dplyr", "broom")

for (pkg in needed_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg, repos = "https://cloud.r-project.org")
  }
}

# Project paths are relative to the repository root. The script can be run
# either from the repository root or from the analysis/ folder.
project_root <- if (file.exists(file.path("data", "diabetes.tab.txt"))) {
  "."
} else if (file.exists(file.path("..", "data", "diabetes.tab.txt"))) {
  ".."
} else {
  stop("Run this script from the repository root or from analysis/. ",
       "Could not find data/diabetes.tab.txt.")
}

data_file  <- file.path(project_root, "data", "diabetes.tab.txt")
figure_dir <- file.path(project_root, "figures")
dir.create(figure_dir, showWarnings = FALSE)

# Save a ggplot figure in both PDF and PNG format.
# Only the figures used in the report and presentation are saved to figures/;
# other plots are drawn in the session for exploration.
save_figure <- function(plot, name, width, height) {
  ggsave(file.path(figure_dir, paste0(name, ".pdf")), plot,
         width = width, height = height)
  ggsave(file.path(figure_dir, paste0(name, ".png")), plot,
         width = width, height = height, dpi = 300)
```

```r
}

# Save several ggplot figures arranged side by side as one panel figure
# (PDF and PNG), using the grid package that ships with R.
save_panel <- function(plots, name, ncol, width, height) {
  draw_panel <- function() {
    grid::grid.newpage()
    n_row <- ceiling(length(plots) / ncol)
    grid::pushViewport(grid::viewport(layout = grid::grid.layout(n_row, ncol)))
    for (i in seq_along(plots)) {
      print(plots[[i]], vp = grid::viewport(
        layout.pos.row = ceiling(i / ncol),
        layout.pos.col = ((i - 1) %% ncol) + 1
      ))
    }
  }
  pdf(file.path(figure_dir, paste0(name, ".pdf")), width = width, height = height)
  draw_panel()
  dev.off()
  png(file.path(figure_dir, paste0(name, ".png")),
      width = width, height = height, units = "in", res = 300)
  draw_panel()
  dev.off()
}

library(ggplot2)
library(dplyr)
library(broom)

# Make printed numbers a little easier to read.
options(digits = 4)


# ============================================================================ #
# QUESTION 1. UNDERSTAND AND PREPARE THE DATA
# ============================================================================ #

# A local copy of the dataset is stored in data/ (path set in section 0).
# The original public source is:
# https://www4.stat.ncsu.edu/~boos/var.select/diabetes.tab.txt

# Read the supplied tab-separated file.
# header = TRUE means that the first row contains the column names.
diabetes <- read.table(data_file, header = TRUE, sep = "\t")

# Convert column names to lower case so they are easier to type consistently.
names(diabetes) <- tolower(names(diabetes))

# The source dataset uses 'y' as the disease-progression outcome.
# Rename it to something clearer for the rest of the analysis.
if ("y" %in% names(diabetes)) {
  diabetes <- diabetes %>% rename(progression = y)
}

# Look at the first few rows.
head(diabetes)

# Number of rows and columns.
```

```r
dim(diabetes)

# Variable names.
names(diabetes)

# Structure and data types.
str(diabetes)

# The variable 'sex' is numerically coded in the source data, but it represents
# categories rather than a continuous measurement. Check the values present,
# then convert it to a factor so R treats it as categorical in later analyses.
unique(diabetes$sex)
diabetes$sex <- factor(diabetes$sex)

# Count missing values in each variable.
colSums(is.na(diabetes))

# Total number of missing values.
sum(is.na(diabetes))

# Check for duplicated rows.
sum(duplicated(diabetes))

# Basic summary of every variable.
summary(diabetes)

# Additional data-quality checks.

# Number of observations in each sex category.
table(diabetes$sex)

# Minimum and maximum of every numeric variable, to spot implausible values.
sapply(diabetes %>% select(where(is.numeric)), range)

# Number of distinct values per variable (very few distinct values can reveal
# rounded or discrete measurements).
sapply(diabetes, function(x) length(unique(x)))

# Mean and standard deviation of each numeric variable, to see whether the
# predictors are on their original scale or have been standardised.
sapply(diabetes %>% select(where(is.numeric)), function(x) c(mean = mean(x), sd = sd(x)))

# ============================================================================ #
# QUESTION 2. EXPLORE THE DATA
# ============================================================================ #

# A helper function for common descriptive statistics.
describe_numeric <- function(x) {
  c(
    n = sum(!is.na(x)),
    mean = mean(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    sd = sd(x, na.rm = TRUE),
    min = min(x, na.rm = TRUE),
    q1 = quantile(x, 0.25, na.rm = TRUE),
    q3 = quantile(x, 0.75, na.rm = TRUE),
    max = max(x, na.rm = TRUE),
    iqr = IQR(x, na.rm = TRUE)
```

```r
  )
}

# Descriptive statistics for the outcome.
describe_numeric(diabetes$progression)

# Descriptive statistics for BMI.
describe_numeric(diabetes$bmi)

# Descriptive statistics for age.
describe_numeric(diabetes$age)

# A compact descriptive table for all numeric variables.
# You do not need to put the whole table in your report.
descriptive_table <- diabetes %>%
  summarise(across(
    where(is.numeric),
    list(
      mean = ~mean(.x, na.rm = TRUE),
      median = ~median(.x, na.rm = TRUE),
      sd = ~sd(.x, na.rm = TRUE),
      min = ~min(.x, na.rm = TRUE),
      max = ~max(.x, na.rm = TRUE)
    )
  ))

print(descriptive_table)

# The same information in a long, easier-to-read layout (one row per variable),
# with quartiles and IQR added.
descriptive_long <- diabetes %>%
  select(where(is.numeric)) %>%
  sapply(describe_numeric) %>%
  t() %>%
  round(2)

print(descriptive_long)

# Progression summarised separately for the two sex codes.
diabetes %>%
  group_by(sex) %>%
  summarise(
    n = n(),
    mean = mean(progression),
    median = median(progression),
    sd = sd(progression),
    iqr = IQR(progression)
  )

# Unusual observations: count values outside the usual boxplot fences
# (below Q1 - 1.5*IQR or above Q3 + 1.5*IQR) for each numeric variable.
# This flags observations for inspection; it is not a rule for removing them.
count_boxplot_outliers <- function(x) {
  fences <- quantile(x, c(0.25, 0.75)) + c(-1.5, 1.5) * IQR(x)
  c(below = sum(x < fences[1]), above = sum(x > fences[2]))
}

diabetes %>%
```

```r
  select(where(is.numeric)) %>%
  sapply(count_boxplot_outliers)

# Simple skewness (third standardised moment): about 0 means symmetric,
# clearly positive means a long right tail.
skewness <- function(x) mean((x - mean(x))^3) / sd(x)^3

diabetes %>%
  select(where(is.numeric)) %>%
  sapply(skewness) %>%
  round(2)

# Distribution of diabetes progression.
plot_progression_hist <- ggplot(diabetes, aes(x = progression)) +
  geom_histogram(bins = 25, colour = "white") +
  labs(
    title = "Distribution of diabetes progression",
    x = "Progression measure",
    y = "Count"
  ) +
  theme_minimal()

# Distribution of BMI.
plot_bmi_hist <- ggplot(diabetes, aes(x = bmi)) +
  geom_histogram(bins = 25, colour = "white") +
  labs(
    title = "Distribution of BMI",
    x = "BMI",
    y = "Count"
  ) +
  theme_minimal()

# One additional useful visualisation: boxplot of progression.
plot_progression_box <- ggplot(diabetes, aes(y = progression)) +
  geom_boxplot() +
  labs(
    title = "Diabetes progression: boxplot",
    y = "Progression measure",
    x = NULL
  ) +
  theme_minimal() +
  theme(axis.text.x = element_blank(), axis.ticks.x = element_blank())

# Additional visualisation: progression for the two sex codes.
# The numeric codes 1 and 2 are kept as labels because the supplied files do not
# say which code is female or male.
plot_progression_by_sex <- ggplot(diabetes, aes(x = sex, y = progression)) +
  geom_boxplot() +
  labs(
    title = "Diabetes progression by sex code",
    x = "Sex (coded 1 and 2)",
    y = "Progression measure"
  ) +
  theme_minimal()

# You may replace or supplement the boxplot with another plot that you find
# more informative. For example, you could examine age, blood pressure or s5.
```

```r
# ============================================================================ #
# QUESTION 3. INVESTIGATE RELATIONSHIPS
# ============================================================================ #

# Keep only genuinely numerical measurement variables for Pearson correlations.
# 'sex' has already been converted to a factor, so it is excluded automatically.
numeric_data <- diabetes %>% select(where(is.numeric))

# Correlation matrix.
cor_matrix <- cor(numeric_data, use = "complete.obs", method = "pearson")
round(cor_matrix, 2)

# Correlations of all numeric variables with progression.
# The outcome itself will appear with correlation 1, so focus on the predictors.
progression_correlations <- sort(
  cor_matrix[, "progression"],
  decreasing = TRUE
)
print(progression_correlations)

# Labelled heatmap of the correlation matrix (ggplot2), saved as PNG and PDF.
# Each cell shows the Pearson correlation; sex is not included.
variable_order <- colnames(cor_matrix)

cor_long <- expand.grid(
  row_variable = variable_order,
  col_variable = variable_order,
  stringsAsFactors = FALSE
)
cor_long$correlation <- cor_matrix[cbind(cor_long$row_variable,
                                         cor_long$col_variable)]
cor_long$row_variable <- factor(cor_long$row_variable, levels = rev(variable_order))
cor_long$col_variable <- factor(cor_long$col_variable, levels = variable_order)

plot_correlation_heatmap <- ggplot(
  cor_long, aes(x = col_variable, y = row_variable, fill = correlation)
) +
  geom_tile(colour = "white") +
  geom_text(aes(label = sprintf("%.2f", correlation)), size = 3) +
  scale_fill_gradient2(
    low = "#2166AC", mid = "white", high = "#B2182B",
    midpoint = 0, limits = c(-1, 1), name = "Pearson r"
  ) +
  labs(
    title = "Correlation matrix of numerical variables",
    x = NULL,
    y = NULL
  ) +
  theme_minimal() +
  theme(panel.grid = element_blank())

save_figure(plot_correlation_heatmap, "correlation_heatmap", width = 7, height = 6)

# Scatterplot 1: BMI and progression.
plot_bmi_scatter <- ggplot(diabetes, aes(x = bmi, y = progression)) +
  geom_point(alpha = 0.65) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
```

```r
    title = "Progression versus BMI",
    x = "BMI",
    y = "Progression measure"
  ) +
  theme_minimal()

# Scatterplot 2: s5 and progression.
# s5 was chosen because, after BMI, it has the strongest correlation with
# progression among the predictors (see progression_correlations).
plot_s5_scatter <- ggplot(diabetes, aes(x = s5, y = progression)) +
  geom_point(alpha = 0.65) +
  geom_smooth(method = "lm", se = TRUE) +
  labs(
    title = "Progression versus s5",
    x = "s5 (serum measurement)",
    y = "Progression measure"
  ) +
  theme_minimal()

# Panel figures used in the report.
save_panel(
  list(plot_progression_hist, plot_bmi_hist, plot_progression_by_sex),
  "panel_distributions", ncol = 3, width = 10, height = 3.6
)

save_panel(
  list(plot_bmi_scatter, plot_s5_scatter),
  "panel_scatterplots", ncol = 2, width = 10, height = 4
)

# ============================================================================ #
# QUESTION 4. BUILD AND INTERPRET REGRESSION MODELS
# ============================================================================ #

# ---- Simple linear regression ------------------------------------------------

# BMI is used as the starting predictor because it is meaningful and commonly
# examined in this dataset. You may choose another predictor if you justify it.
simple_model <- lm(progression ~ bmi, data = diabetes)

# Standard model summary.
summary(simple_model)

# Tidy coefficient table, including confidence intervals.
tidy(simple_model, conf.int = TRUE)

# Model-level statistics such as R-squared and adjusted R-squared.
glance(simple_model)

# Coefficients for writing the fitted equation.
coef(simple_model)

# Comparison model: the same simple regression using s5 as the predictor,
# to see how it differs from BMI.
simple_model_s5 <- lm(progression ~ s5, data = diabetes)

summary(simple_model_s5)
```

```r
tidy(simple_model_s5, conf.int = TRUE)

simple_model_comparison <- bind_rows(
  glance(simple_model) %>% mutate(model = "Simple: BMI"),
  glance(simple_model_s5) %>% mutate(model = "Simple: s5")
) %>%
  select(model, r.squared, adj.r.squared, sigma)

print(simple_model_comparison)

# Residuals versus fitted values for the simple BMI model.
diagnostic_data_simple <- augment(simple_model)

plot_residuals_simple <- ggplot(diagnostic_data_simple, aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.65) +
  geom_hline(yintercept = 0, linetype = 2) +
  geom_smooth(se = FALSE) +
  labs(
    title = "Residuals versus fitted values: simple model (BMI)",
    x = "Fitted values",
    y = "Residuals"
  ) +
  theme_minimal()

# ---- Multiple regression -------------------------------------------------

# Three candidate predictor sets were chosen from the correlation results.
# Model A: the two predictors most strongly correlated with progression.
multiple_model_a <- lm(progression ~ bmi + s5, data = diabetes)

# Model B: model A plus blood pressure (the starter's three-predictor model).
multiple_model_b <- lm(progression ~ bmi + bp + s5, data = diabetes)

# Model C: model B plus sex, entered as a categorical variable (factor).
multiple_model_c <- lm(progression ~ bmi + bp + s5 + sex, data = diabetes)

# Models A, B and C were all fitted and compared. Model A is kept as the
# parsimonious model and model B as the fuller model; the diagnostics and
# uncertainty analysis in Question 5 are run for both. Model C (adding sex)
# gave only a small improvement and is mentioned in the limitations only.
parsimonious_model <- multiple_model_a
fuller_model <- multiple_model_b

summary(multiple_model_a)
summary(multiple_model_b)
summary(multiple_model_c)

tidy(multiple_model_a, conf.int = TRUE)
tidy(multiple_model_b, conf.int = TRUE)
tidy(multiple_model_c, conf.int = TRUE)

# Compare the models using measures taught in the course.
# R-squared shows explained variation, while adjusted R-squared, AIC and BIC
# take model complexity into account (lower AIC/BIC is better).
model_comparison <- bind_rows(
  glance(simple_model) %>% mutate(model = "Simple: BMI"),
  glance(multiple_model_a) %>% mutate(model = "A: BMI + S5"),
  glance(multiple_model_b) %>% mutate(model = "B: BMI + BP + S5"),
```

```r
  glance(multiple_model_c) %>% mutate(model = "C: BMI + BP + S5 + sex")
) %>%
  select(model, r.squared, adj.r.squared, sigma, AIC, BIC)

print(model_comparison)

# The models are nested (A within B within C), so F-tests can check whether
# each added predictor improves the fit by more than chance would suggest.
anova(multiple_model_a, multiple_model_b, multiple_model_c)

# ============================================================================ #
# QUESTION 5. EVALUATE THE ANALYSIS
# ============================================================================ #

# ---- Residuals versus fitted values ------------------------------------

# augment() creates fitted values and residuals in a convenient data frame.
# The same plot is drawn for the parsimonious (A) and fuller (B) models.
plot_residuals_for_model <- function(model, model_label) {
  ggplot(augment(model), aes(x = .fitted, y = .resid)) +
    geom_point(alpha = 0.65) +
    geom_hline(yintercept = 0, linetype = 2) +
    geom_smooth(se = FALSE) +
    labs(
      title = paste("Residuals versus fitted values:", model_label),
      x = "Fitted values",
      y = "Residuals"
    ) +
    theme_minimal()
}

plot_residuals_parsimonious <- plot_residuals_for_model(
  parsimonious_model, "model A (BMI + S5)"
)
plot_residuals_fuller <- plot_residuals_for_model(
  fuller_model, "model B (BMI + BP + S5)"
)

save_figure(plot_residuals_parsimonious, "residuals_vs_fitted_model_a",
            width = 6, height = 4)
save_figure(plot_residuals_fuller, "residuals_vs_fitted_model_b",
            width = 6, height = 4)

# Additional residual and influence checks for models A and B.
#  - residual SD within thirds of the fitted values (a check on constant spread);
#  - residual skewness (a check on symmetry);
#  - Cook's distance and leverage (a check on influential observations).
# Observations above 4/n in Cook's distance are only a rule-of-thumb flag.
check_residuals <- function(model, model_label) {
  res <- residuals(model)
  fit <- fitted(model)
  fitted_thirds <- cut(fit, quantile(fit, 0:3 / 3), include.lowest = TRUE,
                       labels = c("lowest third", "middle third", "highest third"))
  cooks <- cooks.distance(model)

  list(
    model = model_label,
    residual_sd_by_fitted_third = round(tapply(res, fitted_thirds, sd), 1),
```

```
    residual_range = round(range(res), 1),
    residual_skewness = round(skewness(res), 2),
    max_cooks_distance = round(max(cooks), 3),
    n_cooks_above_4_over_n = sum(cooks > 4 / length(cooks)),
    max_leverage = round(max(hatvalues(model)), 3)
  )
}

check_residuals(parsimonious_model, "Model A: BMI + S5")
check_residuals(fuller_model, "Model B: BMI + BP + S5")

# Look for obvious structure rather than trying to prove that every assumption
# is perfectly satisfied. A strong curve or a clear change in spread would be
# warning signs for a simple linear model.


# ---- Multicollinearity using VIF -------------------------------------------

# VIF can be calculated without installing another package.
manual_vif <- function(model) {
  X <- model.matrix(model)[, -1, drop = FALSE]  # remove intercept

  vif_values <- sapply(seq_len(ncol(X)), function(i) {
    target <- X[, i]
    others <- X[, -i, drop = FALSE]

    if (ncol(others) == 0) {
      return(1)
    }

    r2 <- summary(lm(target ~ others))$r.squared
    1 / (1 - r2)
  })

  setNames(vif_values, colnames(X))
}

# VIF for the parsimonious (A) and fuller (B) models, and for model C.
manual_vif(parsimonious_model)
manual_vif(fuller_model)
manual_vif(multiple_model_c)

# There is no single universal VIF cut-off. Interpret the values together with
# the correlation matrix and with the purpose of the model.


# ---- Statistical uncertainty -----------------------------------------------

# A tidy table with estimates, standard errors, t-values, p-values and 95%
# confidence intervals.
coef_table_parsimonious <- tidy(parsimonious_model, conf.int = TRUE)
coef_table_fuller <- tidy(fuller_model, conf.int = TRUE)
print(coef_table_parsimonious)
print(coef_table_fuller)

# You can also obtain the confidence intervals directly.
confint(parsimonious_model, level = 0.95)
confint(fuller_model, level = 0.95)
```

```
# ============================================================================ #
# QUESTION 6. DRAW APPROPRIATE CONCLUSIONS
# ============================================================================ #

# No new code is required for this question. The conclusions, limitations and
# discussion of causality are in report/analytical_report.pdf.

##############################################################################
# END OF SCRIPT
##############################################################################
```